

Week 6 • Routing, pages, and going live

Technical Interaction Design • ITU • Autumn 2026

Built 11 September 2026 • commit bc3510e

Latest version: <https://itu-tid.github.io/lecture-notes-pdf/Week-06.pdf>

CONTENTS

Brief History of Web App Architectures

- What is a server?
- Three generations of web applications
- How do we handle multiple pages if we only have one?

Routing in Single Page Applications (SPA)

- Routing in React
- Example routing with `react-router-dom`
- Notes
- Read More

Protecting routes if a user is not logged in

- With a Higher-Order Component
- With Two Different Routers

Brief History of Web App Architectures

Source: [Lectures/Technical/Routing/Web-App-Architectures.md](#) — something unclear? [Open an issue](#), or send a pull request.

What is a server?

A server can refer both to a device in a datacenter but also a special kind of process

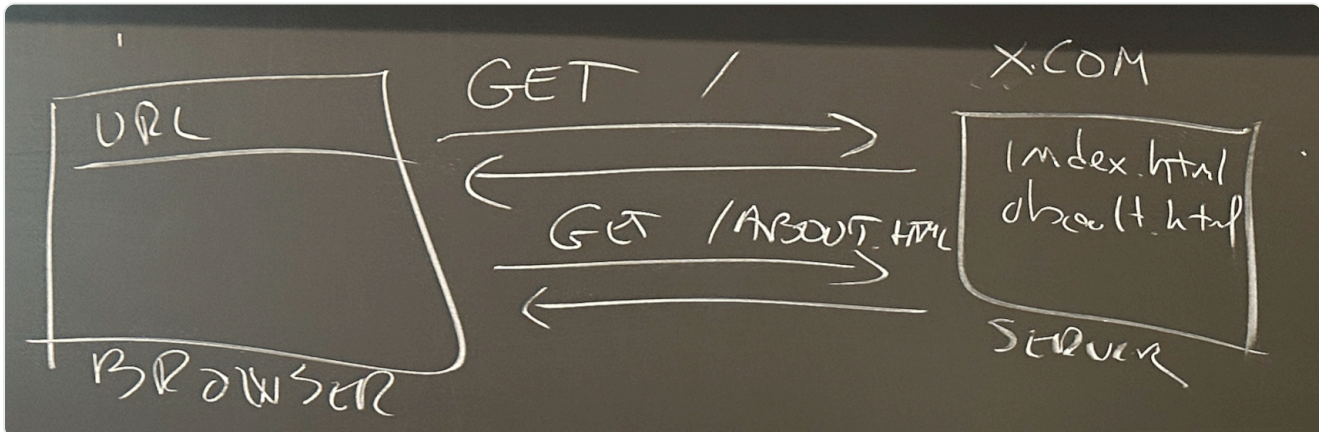
- Example HW server: *our server is an AMD machine with 64GB of RAM*
- Example of SW server: *the Apache web server listening on port 80*

To eliminate the confusion we are going to refer to **web server** or **machine** when we talk about the two types.

Something unclear in this section? [Open an issue](#).

Three generations of web applications

First generation of web applications were simple HTML pages

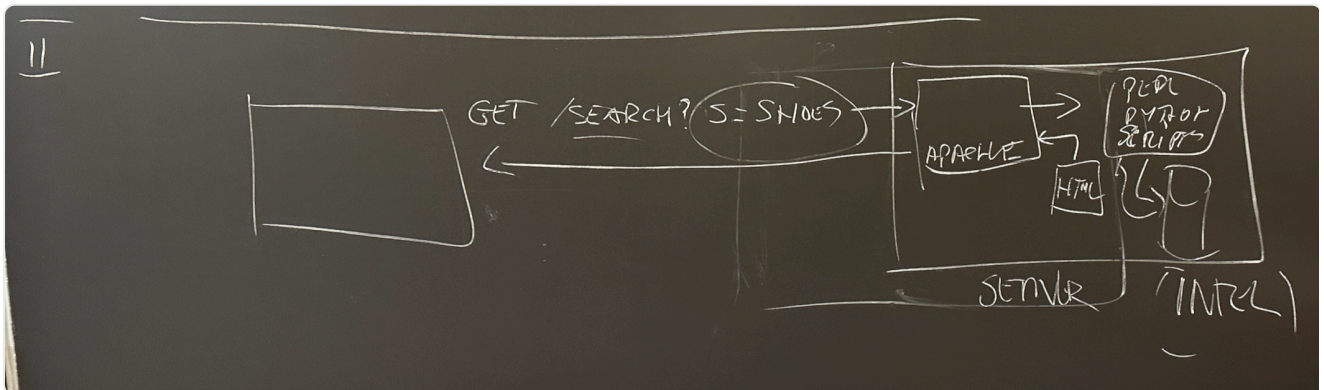


The web server was simply sending a file back

- How?
 - server parses the URL
 - then returns page

2nd generation web application uses scripts on the server to generate pages on the fly

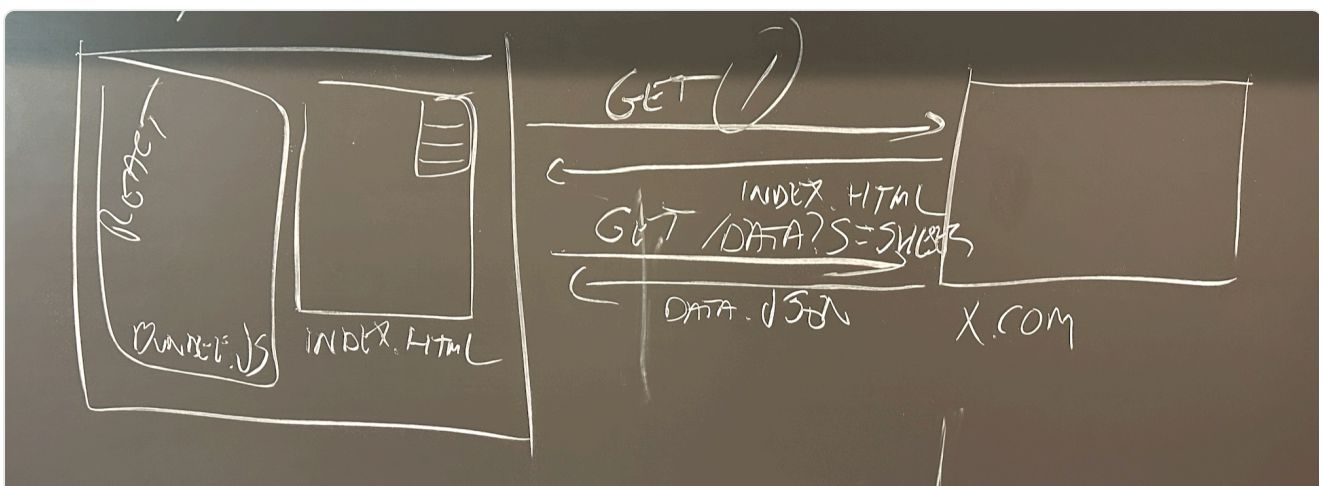
the example in the image below shows how different pages are generated when the client (browser) sends a request by using a parameterized GET request (e.g. `GET /search?term=shoes`)



- server calls the right program, which
 - generates the HTTP response
 - returns it to the server
 - server sends the response forward to the client
- for the client it looks like it asked for a file and it got a file; but in practice the content was **dynamically generated**
- as opposed to **static content**

3rd generation are Single Page Applications

The example in the image below shows that the server now sends both an `index.html` and a React-generated `bundle.js` that's imported from it to the client. The whole application logic is in the `bundle.js` file and is generated by a compilation process on the server.



- The SPA is only calling the web server to ask for data
- The UI is rendered on the client with React

Historical Note

SPA as an architecture was popularized by Google and Facebook via their respective technologies: AngularJS and React.

- **2009-2010:** AngularJS development begins and releases
- **2011:** React developed internally at Facebook
- **2012:** AngularJS 1.0 released; React used on Instagram
- **2013:** React open-sourced by Facebook
- **2014+:** Both frameworks gain massive adoption

React is just one of many frameworks that use the SPA architecture

Something unclear in this section? [Open an issue.](#)

How do we handle multiple pages if we only have one?

Exam Questions

1. Describe the three generations of web application architectures.
2. What is a Single Page Application (SPA) and what are its characteristics?

Something unclear in this section? [Open an issue.](#)

Routing in Single Page Applications (SPA)

Source: [Lectures/Technical/Router/Router-in-React.md](#) — something unclear? [Open an issue](#), or send a pull request.

Where is routing traditionally implemented in web applications?

- On the server side

However, when you have a SPA?

- You don't want to go to the server for the pages, but generate them locally
- So the routing has to be done on the client side

Why would it be bad if we didn't have URLs anymore in our SPA web applications?

- Usability principle? (conventions that are familiar to the user)
- Deep linking
- Browser functionality

How do we implement routing in SPAS?

- On the client side (i.e. in the browser)
- Every URL request is intercepted by the our SPA

Routing in React

Is Routing the responsibility of React?

- You'd think so... but, nope. React does not care
- React is responsible with the rendering of components
- Routing has to be implemented by a 3rd party library

How would a 3rd party library work?

- **Intercepting the intent of navigating to a different page** and rendering the corresponding page
- How can it intercept?

How to find a routing library?

- Look on `npm`
- Choose the most popular
- Why is this a good idea?
 - popularity is proportional to support
 - *many eyes catch all the bugs*

Something unclear in this section? Open an issue.

Example routing with `react-router-dom`

1. Basic Routing Setup

- Install `react-router-dom` via npm/yarn
- Wrap the app with `<BrowserRouter>`
- Use `<Routes>` and `<Route>` to define routes

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

2. Navigation

- Use `<Link>` for client-side navigation (avoids full page reloads)
- This is as opposite to `<a>` elements - who go to the server and trigger a full page re-render

```
import React from 'react';
import { Link } from 'react-router-dom';
const Header = () => {
  return (
    <div className="App">
      <Link to="/" > Home </Link>
      <Link to="/about" > About </Link>
      <a href="/about">dont' use this!</a>
    </div>
  );
};
```

When you need to change URL url from JS

- Use the `useNavigate` hook for programmatic navigation

```
import React from "react"
import {useNavigate} from "react-router-dom"

export default function Profile() {
  let navigate = useNavigate()
  return (
    <div>
      <h2> Go to profile </h2>
      <button onClick={()=>{ navigate("/about")}}> About
    </button>
    </div>
  );
};
```

Dynamic Routes

- Sometimes you want to pass parameters to the url, e.g. `/users/:id`
- Use `:param` in the path to define dynamic segments

```
// Route definition
<Route path="/users/:userId" element={<UserProfile />} />
```

- Access the parameter with `useParams` hook


```
// Accessing the parameter
function UserProfile() {
  const { userId } = useParams();
  return <div>User ID: {userId}</div>;
}
```

Exact vs. Partial Matching

Partial matching as default

- By default, a route uses partial matching, it will match if the beginning of the URL matches the path

```
<Route path="/about" component={About} />
```

- This will match `/about`, `/about/team`, `/about/us`, etc.

Exact matching as an option

- When you use the **exact** prop on a `<Route>`, the path must match the entire URL for the route to render

```
<Route exact path="/about" component={About} />
```

- This will only match `/about`.
- It will not match `/about/team`, `/about/us`, etc.
- Use this when you want a component to render for a group of related routes (e.g. sidebar for all the logged in routes)

Nested routes

Most often than not, you will want to have nested routes.

- you can define them inside of each other
- you can use the `<Outlet>` element to render the children elements inside of the layout of the main

```
// Parent route
<Route path="/dashboard" element={<DashboardLayout />}>
  <Route index element={<DashboardHome />} />
  <Route path="stats" element={<DashboardStats />} />
</Route>

// DashboardLayout.jsx
function DashboardLayout() {
  return (
    <div>
      <title> The greatest dashboard </title>
      <Sidebar />
      <Outlet /> { /* Child routes render here */}
    </div>
  );
}
```

- Note the `index` - that this is what gets rendered inside of the `<Outlet>`

Protected Routes

- How to protect routes (e.g., redirect unauthenticated users).
- Use `<Navigate>` for redirects
- Create wrapper component for auth checks

```
// Define a wrapper component
function PrivateRoute({ children }) {
  const isAuthenticated = checkAuth(); // Your auth logic
  return isAuthenticated ? children : <Navigate to="/login" />;
}

// Usage
<Route
  path="/profile"
  element={
    <PrivateRoute>
      <Profile />
    </PrivateRoute>
  }
/>
```

URL search parameters

- Search params are query strings that can exist appended at the end of your URL, e.g.

```
/dashboard?sort=name&filter=active  
/profile?tab=settings  
/products?page=2&category=electronics
```

- To access them from within the appropriate component you have to use the `useSearchParams` hook

```
import { useSearchParams } from 'react-router-dom';  
  
function DashboardHome() {  
  const [searchParams, setSearchParams] = useSearchParams();  
  
  const sort = searchParams.get('sort'); // 'name'  
  const filter = searchParams.get('filter'); // 'active'  
  
}
```

Handling unknown routes

- Use a wildcard route (*) to catch all unmatched paths

```
<Routes>  
  <Route path="/" element={<Home />} />  
  <Route path="*" element={<NotFound />} />  
</Routes>
```

An example with Navbar and Sidebar

App.js

```
import { Routes, Route } from 'react-router-dom';
import NavBar from './components/NavBar';
import About from './components/About';
import MainLayout from './components/MainLayout';
import Feed from './components/Feed';
import Profile from './components/Profile';

function App() {
  return (
    <
      /* Top navigation (shared everywhere) */
      <NavBar />

      /* Routes */
      <Routes>
        <Route path="/about" element={<About />} />
        <Route path="/main" element={<MainLayout />}>
          <Route path="feed" element={<Feed />} />
          <Route path="profile" element={<Profile />} />
        </Route>
      </Routes>
    </>
  );
}
```

NavBar.js - top navigation

```
import { Link } from 'react-router-dom';

export default function NavBar() {
  return (
    <nav style={{ background: '#333', color: 'white', padding: '10px' }}>
      <ul style={{ display: 'flex', gap: '20px', listStyle: 'none' }}>
        <li><Link to="/about" style={{ color: 'white' }}>About</Link></li>
        <li><Link to="/main/feed" style={{ color: 'white' }}>Feed</Link></li>
        <li><Link to="/main/profile" style={{ color: 'white' }}>Profile</Link></li>
      </ul>
    </nav>
  );
}
```

or better yet, using the `NavLink` from the framework you can do:

```
import { NavLink } from 'react-router-dom';

export default function NavBar() {
  return (
    <nav style={{ background: '#333', color: 'white', padding: '10px' }}>
      <ul style={{ display: 'flex', gap: '20px', listStyle: 'none' }}>
        <li><NavLink to="/about" style={{ color: 'white' }}>About</NavLink></li>
        <li><NavLink to="/main/feed" style={{ color: 'white' }}>Feed</NavLink></li>
        <li><NavLink to="/main/profile" style={{ color: 'white' }}>Profile</NavLink></li>
      </ul>
    </nav>
  );
}
```

and add the following CSS:

```
.active {
  font-weight: bold;
  text-decoration: underline;
}
```

because the framework (`react-router-dom`) automatically adds the `.active` class to the link that matches the current URL.

MainLayout.js - the sidebar

```
import { Outlet } from 'react-router-dom';

export default function MainLayout() {
  return (
    <div style={{ display: 'flex' }}>
      /* Sidebar (shared for /main/* routes) */
      <div style={{ width: '200px', background: '#f0f0f0', padding: '10px' }}>
        <h3>Main Menu</h3>
        <ul style={{ listStyle: 'none', padding: 0 }}>
          <li><a href="/main/feed">Feed</a></li>
          <li><a href="/main/profile">Profile</a></li>
        </ul>
      </div>

      /* Dynamic content for child routes */
      <div style={{ flex: 1, padding: '20px' }}>
        <Outlet /> /* Feed or Profile renders here */
      </div>
    </div>
  );
}
```

Note: you can also use the `NavLink` here to highlight the currently selected element

About.js

```
export default function About() {
  return (
    <div style={{ padding: '20px' }}>
      <h1>About Us</h1>
      <p>This is a standalone page with only the top navigation.</p>
    </div>
  );
}
```

Feed.js and Child.js

```
// Feed.js
export default function Feed() {
  return <h2>Your Feed Content</h2>;
}

// Profile.js
export default function Profile() {
  return <h2>Your Profile Content</h2>;
}
```

Something unclear in this section? Open an issue.

Notes

If the user does a reload on your `/about` page?

The request will go to the server. However, we only have an `index.html` on the server, we do not have an about!

- The solution in this kind of situations is that normally, the web host will redirect all the page-not-found to the `index.html` such that it's still the react app that gets to handle the request. And the react app will parse the url of the page, and handle it again.

But this is only one library... how do we do routing with others

- if you understand the concepts here you will have a much easier time understanding other similar libraries
- the problems described above are the same

Something unclear in this section? Open an issue.

Read More

- [React Router Declarative Mode](#) - the official documentation - note that there's also Data Mode and Framework mode that we didn't talk about

Exam Questions

1. Why is client-side routing necessary in SPAs?
2. What is the difference between `<Link>` and `<a>` tags in React Router?
3. Explain what this routing setup does:

```
<Routes>
  <Route path="/dashboard" element={<DashboardLayout />}>
    <Route index element={<DashboardHome />} />
    <Route path="stats" element={<DashboardStats />} />
  </Route>
</Routes>
```

4. What does this protected route component do?

```
function PrivateRoute({ children }) {
  const isAuthenticated = checkAuth();
  return isAuthenticated ? children : <Navigate to="/login" />;
}
```

5. How do you access URL parameters in React Router?

```
// Route: <Route path="/users/:userId" element={<UserProfile />} />
// URL: /users/123
```

6. How do you access query string parameters?

```
// URL: /dashboard?sort=name&filter=active
```

7. What happens if a user reloads the page on `/about` in an SPA?

Something unclear in this section? Open an issue.

Protecting routes if a user is not logged in

Source: [Lectures/Technical/Routing/Protecting-Routes.md](#) — something unclear? [Open an issue](#), or send a pull request.

Example: How to improve the code below?

```
// App.js
// ...
return (
  <Routes>
    <Route path="/" element={<Login />} />
    <Route path="/event-detail/:eventId" element={<EventDetail />} />    </Routes>
  );

// EventDetailPage.js
const EventDetailPage = () => {

  // ...

  const currentUser = Parse.User.current();

  // ...

  if (!currentUser) {
    return (
      <div className="error-container">
        <p>Error: User not logged in.</p>
      </div>
    );
  }
}
```

With a Higher-Order Component

Creating another React component named `RequireAuth` that redirects to the Login page if the user is not logged in.

```
import Parse from "../services/Parse";
import Login from "../pages/LoginPage";

export default function RequireAuth({children}) {
  const currentUser = Parse.User.current();

  if (!currentUser) {
    return <Login />;
  }

  return children;
}
```

This component can be then used in the router like this:

```
// ...
<Route path="/event-detail/:eventId" element={
  <RequireAuth>
    <EventDetail />
  </RequireAuth>}
/>
```

Something unclear in this section? [Open an issue](#).

With Two Different Routers

```
// inside App.js or wherever you do your routing
const currentUser = Parse.User.current();

if (!currentUser) {
  // return simplified version of routing
  return (
    <Routes>
      <Route path="/" element={<Login />} />
      <Route path="/signup" element={<SignUpPage />} />

      <Route path="*" element={<Navigate to="/" />} />
    </Routes>
  )
}

// Return the routing for when the a user is logged in
return (
  <Routes>
    <Route path="/home" element={<HomePage />} />
    <Route path="/create-event" element={<CreateEvent />} />
    <Route path="/event-detail/:eventId" element={<EventDetail />} />
  </Routes>
);
```

Something unclear in this section? Open an issue.